

Regex Cheat Sheet

Quick reference for regular expression patterns - now with Common patterns and Flags/Modifiers

 Search patterns, descriptions, or examples...

All Categories

Common

Character classes

Anchors

Escaped characters

Groups & Lookaround

Quantifiers & Alternation

Flags/Modifiers

Substitution

Common

Most Used

[abc] A single character of: a, b or c

Example: [abc] matches a, b, or c



[^abc] A character except: a, b or c

Example: [^abc] matches d, e, f...



[a-z] A character in the range: a-z

Example: [a-z] matches lowercase letters



[^a-z] A character not in the range: a-z

Example: [^a-z] matches uppercase, digits, symbols



[a-zA-Z] A character in the range: a-z or A-Z

Example: [a-zA-Z] matches any letter



. Any single character

Example: a.c matches abc, a1c, a@c



a|b Alternate - match either a or b

Example: `cat|dog` matches `cat` or `dog`



`\s` Any whitespace character

Example: `\s+` matches spaces, tabs, newlines



`\S` Any non-whitespace character

Example: `\S+` matches visible characters



`\d` Any digit

Example: `\d+` matches `123`



`\D` Any non-digit

Example: `\D+` matches `abc`



`\w` Any word character

Example: `\w+` matches `hello123`



`\W` Any non-word character

Example: `\W+` matches `@#$`



`(?:...)` Non-capturing group

Example: `(?:abc)+` matches `abc` sequences



`(...)` Capturing group

Example: `(\d+)` captures numbers



`a?` Zero or one of `a`

Example: `ab?c` matches `ac`, `abc`



`a*` Zero or more of `a`

Example: `ab*c` matches `ac`, `abc`, `abbc`



`a+` One or more of `a`

Example: `ab+c` matches `abc`, `abbc`



a{3} Exactly 3 of a

Example: \d{3} matches 123



a{3,} 3 or more of a

Example: \d{3,} matches 123, 1234...



a{3,6} Between 3 and 6 of a

Example: \d{3,6} matches 123, 1234, 12345, 123456



^ Start of string

Example: ^hello matches hello at start



\$ End of string

Example: world\$ matches world at end



\b A word boundary

Example: \bcat\b matches cat as whole word



\B Non-word boundary

Example: \Bcat\b matches cat within word



Character classes

\w \d \s word, digit, whitespace

Example: \w+ \d+ \s+ matches word123 space



\W \D \S not word, digit, whitespace

Example: \W+ matches @#\$



[a-g] character between a & g

Example: [a-g] matches a, b, c, d, e, f, g



[:alnum:] Letters and digits



Example: `[:alnum:]` matches alphanumeric characters

`[:alpha:]` Letters

Example: `[:alpha:]` matches alphabetic characters



`[:ascii:]` ASCII codes 0-127

Example: `[:ascii:]` matches ASCII characters



`[:blank:]` Space or tab only

Example: `[:blank:]` matches space and tab



`[:cntrl:]` Control characters

Example: `[:cntrl:]` matches control characters



`[:digit:]` Decimal digits

Example: `[:digit:]` matches 0-9



`[:graph:]` Visible characters (not space)

Example: `[:graph:]` matches printable non-space



`[:lower:]` Lowercase letters

Example: `[:lower:]` matches a-z



`[:print:]` Visible characters

Example: `[:print:]` matches printable characters



`[:punct:]` Visible punctuation characters

Example: `[:punct:]` matches punctuation



`[:space:]` Whitespace

Example: `[:space:]` matches whitespace characters



`[:upper:]` Uppercase letters

Example: `[:upper:]` matches A-Z



`[:word:]` Word characters

Example: `[[:word:]]` matches word characters



`[[:xdigit:]]` Hexadecimal digits

Example: `[[:xdigit:]]` matches 0-9A-Fa-f



`[[:<:]]` Start of word

Example: `[[:<:]]` matches word start boundary



`[[:>:]]` End of word

Example: `[[:>:]]` matches word end boundary



Anchors

`\G` Start of match

Example: `\G\d+` matches digits from last match position



`^` Start of string

Example: `^hello` matches hello at start of string



`$` End of string

Example: `world$` matches world at end of string



`\A` Start of string

Example: `\Ahello` matches hello at very start



`\Z` End of string

Example: `world\Z` matches world at very end



`\z` Absolute end of string

Example: `end\z` matches end at absolute end



`\b` A word boundary

Example: `\bword\b` matches whole word



\B Non-word boundary

Example: \Bcat\b matches cat within word



Escaped characters

**\. * ** escaped special characters

Example: \. matches literal dot



\+ reserved characters

Example: \+ matches literal plus sign



\000 octal escape

Example: \000 matches null character



\xFF hexadecimal escape

Example: \xFF matches character with hex value FF



\uFFFF unicode escape

Example: \u0041 matches character A



\u{FFFF} extended unicode escape

Example: \u{1F600} matches emoji



\cI control character escape

Example: \cI matches tab character



\t tab

Example: \t matches tab character



\n line feed

Example: \n matches newline character



\v vertical tab



Example: `\v` matches vertical tab

`\f` form feed

Example: `\f` matches form feed character



`\r` carriage return

Example: `\r` matches carriage return



`\0` null

Example: `\0` matches null character



Groups & Lookaround

`(?:...)` Non-capturing group

Example: `(?:abc)+` matches abc sequences



`(...)` Capturing group

Example: `(\d+)` captures numbers



`(?>...)` Atomic group (non-capturing)

Example: `(?>\d+)` matches without backtracking



`(?|...)` Duplicate/reset subpattern group number

Example: `(?|(a)|(b))` resets group numbering



`(?#...)` Comment group

Example: `(?#this is a comment)abc`



`(?'name'...)` Named Capturing Group

Example: `(?'word'\w+)` captures named group



`(?<name>...)` Named Capturing Group

Example: `(?<word>\w+)` captures named group



(?P<name>...) Named Capturing Group

Example: (?P<word>\w+) Python-style named group



(?imsxUJnxx) Inline modifiers

Example: (?i)abc matches case-insensitive



(?imsxUJnxx:...) Localized inline modifiers

Example: (?i:abc) applies case-insensitive to abc only



(?(1)yes|no) Conditional statement

Example: (a)?(?(1)b|c) matches ab or c



RegexBox

Home

Test

Cheat
Sheet

Community

Templates

Login

Sign Up

Example: (? (R)recursive|normal) checks recursion

(?(R#)yes|no) Recursive Conditional statement

Example: (? (R#)recursive|normal) checks specific recursion



(?(R&name)yes|no) Conditional statement

Example: (? (R&name)yes|no) checks named recursion



(?(?=...)yes|no) Lookahead conditional

Example: (? (=?=d)\d+|\w+) matches digits or words



(?(?<=...)yes|no) Lookbehind conditional

Example: (? (?<=@)\w+|\d+) matches based on lookbehind



(?(DEFINE)...) Pre-define patterns before using them

Example: (? (DEFINE)(?<word>\w+))(?&word)



(?R) Recurse entire pattern

Example: \(((?>[^\()]+|(?R))*\) matches nested parentheses



(?1) Match expression defined in capture group 1

Example: (\w+)\s+(?1) matches repeated words



(?+1) Match expression defined in the first relative capture group

Example: (?+1) references relative group



(?&name) Match expression defined in capture group `name`

Example: (?<word>w+)\s+(?&word) matches repeated named groups



(?P=name) Match subpattern `name`

Example: (?P<word>w+)\s+(?P=word) Python-style backreference



(?P>name) Match expression defined in the capture group "{name}"

Example: (?P<word>w+)\s+(?P>word) Python-style subroutine



\1 backreference to group #1

Example: (w)\1 matches doubled letters



(?=...) Positive Lookahead

Example: \d+(?=px) matches numbers before px



(?!...) Negative Lookahead

Example: \d+(?!px) matches numbers not before px



(?<=...) Positive Lookbehind

Example: (?<=)\d+ matches numbers after \$



(?<!...) Negative Lookbehind

Example: (?<!un)happy matches happy not preceded by un



(*ACCEPT) Control verb

Example: (*ACCEPT) forces match to succeed



(*FAIL) Control verb

Example: (*FAIL) forces match to fail



(*MARK:NAME) Control verb

Example: (*MARK:label) marks position for backtracking



Quantifiers & Alternation

a? Zero or one of a

Example: ab?c matches ac, abc



a* Zero or more of a (greedy)

Example: ab*c matches ac, abc, abbc



a+ One or more of a

Example: ab+c matches abc, abbc



a{3} Exactly 3 of a

Example: \d{3} matches 123



a{3,} 3 or more of a

Example: \d{3,} matches 123, 1234...



a{3,6} Between 3 and 6 of a

Example: \d{3,6} matches 123, 1234, 12345, 123456



a*? Lazy quantifier

Example: a*? matches as few a's as possible



a*+ Possessive quantifier

Example: a*+ matches without backtracking



ab|cd match ab or cd

Example: cat|dog matches cat or dog





Global

Example: `/abc/g` matches all occurrences



Multiline

Example: `/^abc/m` matches abc at line start



Case insensitive

Example: `/abc/i` matches ABC, abc, Abc



Ignore whitespace / verbose

Example: Allows comments and whitespace



Single line

Example: `.` matches newline characters



Unicode

Example: Enables Unicode matching



eXtra

Example: Extra features (Perl)



Ungreedy

Example: Makes quantifiers non-greedy by default



Sticky

Example: Matches from `lastIndex` position



hasIndices

Example: Generate indices for substring matches



Substitution



Complete match contents



Example: Replace with entire matched text

\$1 Contents in capture group 1

Example: Replace with first captured group



\$\$ Insert a dollar sign

Example: Replace with literal \$ character



\${foo} Contents in capture group `foo`

Example: Replace with named group 'foo'



\x20 Hexadecimal replacement values

Example: Replace with space character (hex 20)



\x{06fa} Hexadecimal replacement values

Example: Replace with Unicode character U+06FA



\t Insert a tab

Example: Replace with tab character



\r Insert a carriage return

Example: Replace with carriage return character



\n Insert a newline

Example: Replace with newline character



\f Insert a form-feed

Example: Replace with form-feed character



\U Uppercase Transformation

Example: Convert following text to uppercase



\L Lowercase Transformation

Example: Convert following text to lowercase



\E Terminate any Transformation

Example: End case transformation



`${1:+foo:bar}` Conditional replacement



Example: Replace with 'foo' if group 1 exists, else 'bar'

`\[` Insert the escaped literal



Example: Replace with literal [character

RegexBox

An AI-powered regex generator and tester that helps you create, validate, and optimize regular expressions instantly. Explore a growing regex library, share custom patterns, and discover ready-to-use solutions for common use cases. Whether you're a beginner or an experienced developer, our tool makes working with regex faster, smarter, and easier.

Product

- Regex Generator
- Pattern Library
- Tester
- Cheat Sheet
- Community

About

- About Us
- Contact

Legal

- Privacy Policy
- Terms of Service